

Class 3

Tuesday, September 15

The bias-variance tradeoff and train-validation-test splits

Upcoming

Reading (complete by Thursday, September 17): [ISL textbook](#) Chapter 3, if you have not already read it.

Homework (due Thursday, September 17 before class at 12:59 p.m.): Homework 1

Key takeaways

- The generalization performance of a prediction method increases with the bias and variance of the prediction method.
- More flexible prediction methods often reduce bias but increase variance. Finding a good prediction method often involves balancing bias and variance.
- In practice, we can select between and evaluate prediction methods using a train-validation-test split.

Summary notes

Recap: measuring generalization and the optimal regression predictor. Last class, we saw that the ability of a prediction method to *generalize* (i.e. to perform well on new data) can be measured using the *expected test mean square error*. You all also asked some great questions about how to select between different prediction methods (for example the number of neighbors in nearest neighbors) *when we have real data*. Today, we will mostly focus on this practical question.

The bias-variance trade-off. Recall $Y = f(X) + \epsilon$, with mean-zero noise independent of X . The optimal predictor is $f(x) = \mathbb{E}[Y \mid X = x]$. At a fixed input x , let $\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]$ be our average prediction over repeated training datasets. That is, $\bar{f}(x)$ is the predictor we'd get if we had many, many datasets that "looked like" the data we got, we fit our prediction method on each of these datasets, then average the predictions. In practice, we only have one dataset, so we can't actually calculate this decomposition. For a new response $Y_0 = f(x) + \epsilon_0$, independent of the training data,

$$\underbrace{\mathbb{E}_{\mathcal{D}, \epsilon_0}[(Y_0 - \hat{f}_{\mathcal{D}}(x))^2]}_{\text{expected test MSE at } x} = \underbrace{(\bar{f}(x) - f(x))^2}_{\text{squared bias}} + \underbrace{\text{Var}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}(x))}_{\text{variance}} + \underbrace{\text{Var}(\epsilon)}_{\text{irreducible noise}}.$$

- **Bias:** how far our average prediction is from the optimal predictor. Squared bias measures the size of this gap. Often, simple prediction methods have high bias, because they aren't flexible enough to model the unknown function f .
- **Variance:** how sensitive the prediction is to the noise in the training data. When we redraw the whole dataset, variation in the training inputs also contributes. Often, flexible prediction methods have high variance, because they are sensitive to noise in the observed data.
- **Irreducible noise:** even the optimal predictor can't predict ϵ , the noise on the new, unseen data. Its contribution is $\text{Var}(\epsilon)$, as in lecture 2.

The bias-variance decomposition isn't something we can calculate in practice, since we only have one dataset and we don't know f or $\bar{f}$. But the decomposition illustrates the tradeoff we often face when trying to decide whether to apply a complicated, very flexible prediction approach, or a simpler prediction approach.

Train–validate–test splits. We still need a practical tool to let us decide between different prediction methods, and we can't use the bias variance decomposition in practice. A really common way to select between and validate prediction methods in statistics, data science and machine learning is to use a *train-validate-test split*. We discussed this idea briefly last class. The idea is to split the dataset into 3 parts at random (e.g. 60%, 20%, 20% although the percentages used could differ). The first portion of the data, the training data, are used to fit any candidate models. Each of these models is then evaluated on the second (validation) portion of the data. The model that performs best (e.g. has lowest mean squared error) is selected as the prediction method to use. The third portion of data is then used to evaluate the accuracy of this prediction method.

Set	General job	Example: KNN Regression
Training	Build prediction methods	Supply neighbors for each candidate K .
Validation	Select between prediction methods	Compare prediction MSE for different K , choose the lowest.
Test	Evaluate prediction method	Evaluate the final predictor after K is fixed.

Refitting after validation (optional). After choosing a particular prediction method to use (e.g. a value of K), it can be advantageous to combine the training and validation data to build a final predictor, and then evaluate (only) this final predictor on the test set. I'm not aware of complete theory supporting this as an approach, but the intuition is that it's usually better to use more data. And at least the test set mean square error still gives a good (unbiased) estimate of this new predictor's expected test mean squared error, and so we can tell if the predictor we ended up with is any good.

Train-validate–test splits on the Auto dataset. We will now go through what this looks like, using the Auto dataset you are also looking at on the homework as an example.

If we used the same reserved cars both to choose K and to report its performance, what might go wrong?